Project: Image Super-Resolution using Machine Learning

Name: Md. Sifur Rahman
ID: 1620018042
Course Code: CSE 445
Section: 06
Group No: 04

Week 1 (Feb 5 – Feb 11): Dataset Planning and Image Collection

During the first week of the project, my main responsibility was to plan the dataset structure and collect appropriate images for the super-resolution task. Since the project aims to train a machine learning model to reconstruct high-resolution images from low-resolution inputs, we needed a varied collection of images containing different textures, objects, and scenes.

I gathered 100 random images from different internet sources while making sure that the images included enough visual detail, such as edges, textures, and color variations. These features are important for super-resolution tasks because the model must learn how fine details are lost during downscaling and how those details can be reconstructed.

After collecting the images, I arranged them systematically so that they could later be processed and used in the training pipeline.

Week 2 (Feb 12 – Feb 19): Dataset Standardization and Resolution Preparation

During the second week, I worked with another group member to prepare the dataset for machine learning training. Since the images collected from the internet had different dimensions and aspect ratios, they first needed to be standardized.

All images were resized to a consistent resolution of 256 × 256 pixels. These images act as the ground truth high-resolution (HR) images for the model. After preparing the HR images, I created the corresponding low-resolution (LR) images by downscaling them by a factor of four. As a result, every 256 × 256 HR image was converted into a 64 × 64 version.

This process represents a real-world situation in which only a low-resolution image is available and the model attempts to reconstruct a higher-resolution version. The HR images were saved in a folder named HR_256, while the downscaled LR images were stored in LR_x4. This folder structure allowed the training pipeline to easily find the corresponding image pairs.

Week 3 (Feb 20 – Mar 1): Dataset Organization and Training/Test Split

After preparing the HR and LR datasets, I organized them so they could be used efficiently during training. I created a consistent naming convention for every image using numerical IDs, such as 0000, 0001, 0002, and so on. This made it easy to match each low-resolution image with its corresponding high-resolution image.

I then split the dataset into training and testing sets. The first 100 images were used to train the model, while the remaining images were kept for testing and evaluation. This division allows the model to learn from one part of the dataset and then be evaluated on images it has not seen before.

This step was necessary to make sure that the model does not simply memorize the images, but instead learns how to reconstruct high-resolution details from low-resolution inputs.

Week 4 (Mar 2 – Mar 8): Dataset Class Implementation and Data Loading Pipeline

During the fourth week, I built the dataset loading pipeline using PyTorch. I created a custom dataset class that automatically loads matching LR and HR image pairs from their respective dataset folders.

The dataset class also supports patch-based training. Rather than feeding complete images into the model, small patches are randomly taken from both the low-resolution and high-resolution images. This method increases the effective amount of training data and helps the model learn local image features more efficiently.

Along with patch extraction, I added basic data augmentation methods such as horizontal and vertical flipping. These augmentations increase the diversity of the dataset and help reduce overfitting during training.

Finally, I prepared the DataLoader that supplies batches of image patches to the model during training. This component improves training efficiency by automatically loading and shuffling the data.

With the dataset preparation, preprocessing pipeline, and data loading system fully completed, the project is now prepared for the model training phase.